Compte rendu TP 3

1. Configuration du serveur nginx

Le serveur www.tp.local (nginx) présente la chaîne serveur_chaine.crt (certificat du serveur suivi de l'intermédiaire, jamais la racine) et exige un certificat client (mTLS). Les fichiers sont installés dans /etc/nginx/tls/ ; la clé privée a les droits 600. Le nom www.tp.local pointe vers 127.0.0.1 par /etc/hosts. Le fichier de configuration est tp.conf.

Directive	Justification
listen 443 ssl; server_name www.tp.local;	Écoute HTTPS sur le port 443 pour le nom du laboratoire.
ssl_certificate / ssl_certificate_key	Chaîne présentée au client (feuille + intermédiaire) et clé privée correspondante.
ssl_protocols TLSv1.2 TLSv1.3	Interdit SSL et TLS 1.0 / 1.1, obsolètes et vulnérables.
ssl_ciphers ECDHE-ECDSA-…GCM…:CHACHA20-POLY1305	Limite TLS 1.2 aux suites à confidentialité persistante (ECDHE) et à chiffrement authentifié (GCM, ChaCha20-Poly1305). TLS 1.3 n'a que des suites AEAD.
ssl_ecdh_curve X25519:prime256v1:secp384r1	Courbes autorisées pour l'échange de clés, par ordre de préférence.
ssl_prefer_server_ciphers off	Laisse le client choisir parmi des suites toutes jugées sûres, ce qui est la pratique moderne.
ssl_session_tickets off	Évite qu'une clé de ticket de longue durée, non renouvelée, permette de déchiffrer des sessions passées (confidentialité persistante).
ssl_client_certificate racine.crt	Ancre de confiance servant à valider les certificats des clients.
ssl_verify_client on; ssl_verify_depth 2	Refuse tout client sans certificat valide ; profondeur 2 = client → intermédiaire → racine.
ssl_crl crls.pem	Charge les CRL de l'intermédiaire et de la racine pour rejeter les certificats révoqués.
add_header Strict-Transport-Security ... always	HSTS : le navigateur n'utilisera plus que HTTPS pour ce site pendant un an.
return 200 "Bonjour $ssl_client_s_dn"	Affiche le sujet du certificat client, ce qui prouve l'authentification mutuelle.

sudo nginx -t répond syntax is ok et test is successful (visible aussi dans la figure 9). nginx est démarré avec sudo systemctl start nginx ; reload ne fonctionne que s'il est déjà actif.

2. Authentification mutuelle et révocation

Analyse: Bob présente sa chaîne (bob_chaine.crt) et obtient la page. Sans certificat, nginx refuse la requête au niveau HTTP (400). Alice présente un certificat encore valide en dates, mais son numéro de série figure dans la CRL chargée par ssl_crl : nginx le rejette avec l'erreur 23 certificate revoked. La révocation du TP 2 est donc effective en production. Le journal enregistre ces erreurs au niveau info, que j'ai dû activer dans la configuration.

3. Analyse de la poignée de main

Relevés avec openssl s_client -brief (connexion de Bob) : **version TLSv1.3**, **suite TLS_AES_256_GCM_SHA384**, **groupe d'échange X25519** (253 bits), **Verification: OK**, et Verified peername: www.tp.local grâce à -verify_hostname. La connexion a été capturée sur l'interface lo avec tshark (16 paquets) puis ouverte dans Wireshark.

ClientHello. Le client annonce supported_versions (TLS 1.3, TLS 1.2), le nom demandé (SNI) et sa part de clé dans key_share. Ici l'extension fait 1 258 octets : le client propose un échange hybride X25519MLKEM768 (résistant aux ordinateurs quantiques) en plus de x25519. Le champ Version: TLS 1.2 de l'en-tête est un champ hérité, ignoré.

ServerHello. Le serveur choisit TLS_AES_256_GCM_SHA384 et répond avec sa part de clé x25519 ; il retient donc l'échange classique. Tous les messages suivants (Application Data) sont chiffrés avec les clés issues de l'échange.

Pourquoi le certificat du serveur n'est pas visible en TLS 1.3 ? Le ServerHello est le dernier message en clair. EncryptedExtensions, CertificateRequest, Certificate, CertificateVerify et Finished sont envoyés chiffrés, ce qui cache l'identité du serveur à un observateur du réseau. En exportant les clés de session (SSLKEYLOGFILE, keys.log) et en les donnant à tshark, ces messages deviennent lisibles, y compris la requête de certificat du serveur et le certificat de Bob : c'est le mTLS en action. En TLS 1.2, le certificat serait visible en clair.

4. Vérification de la configuration

Un test avec openssl s_client -tls1_1 doit échouer, puisque ssl_protocols n'autorise que TLS 1.2 et 1.3. nmap --script ssl-enum-ciphers -p 443 www.tp.local doit lister uniquement TLSv1.2 (suites ECDHE avec GCM ou ChaCha20) et TLSv1.3, sans suite dépourvue de confidentialité persistante ou de chiffrement authentifié.

[À compléter : coller ici le résultat de tls11.txt et l'extrait de nmap.txt (sections TLSv1.2 / TLSv1.3 et note finale).]

5. Supervision des expirations

surveiller.py lit tous les fichiers .crt d'un dossier, calcule les jours restants et classe chaque certificat : OK (30 jours ou plus), ALERTE (moins de 30 jours), CRITIQUE (moins de 7 jours). Son code de sortie vaut 0, 1 ou 2 selon le pire niveau, ce qui le rend utilisable par un outil de supervision.

6. Renouvellement automatique

renouveler.sh effectue, avec set -euo pipefail : (1) génération d'une nouvelle clé et d'une CSR ; (2) signature par l'intermédiaire ; (3) vérification de la chaîne avec openssl verify avant tout déploiement ; (4) installation, nginx -t puis rechargement sans interruption ; (5) archivage de l'ancien certificat dans archive/<date> avec une fonction de retour arrière en cas d'échec. Sans option, il ne renouvelle que si le certificat expire dans moins de 30 jours (-checkend) ; --force permet le test.

Le numéro de série change (1006 → 1008), la date de fin passe au 28 janvier 2027, l'ancien certificat est conservé dans archive/ et le serveur répond toujours « Bonjour CN=bob@tp.local ». La tâche cron de root lance le script chaque jour à 3 h (0 3 * * *) et journalise dans /var/log/renouveler.log. Elle ne renouvelle qu'en dessous de 30 jours. L'option unique_subject = no de inter.cnf permet de réémettre un certificat avec le même sujet.

7. Questions de réflexion

1. Pourquoi le serveur ne doit-il pas envoyer le certificat racine ?
Le client la possède déjà dans son magasin de confiance : l'envoyer alourdit la poignée de main pour rien. Surtout, elle ne prouve rien : un certificat racine envoyé par le serveur n'est pas une ancre de confiance, car la confiance vient du magasin local. Le serveur envoie donc sa feuille et les intermédiaires.

2. Quel est l'intérêt de désactiver les tickets de session si les clés de tickets ne sont pas renouvelées ?
Un ticket est chiffré avec une clé du serveur. Si cette clé reste longtemps la même et fuit, un attaquant qui a enregistré le trafic peut déchiffrer les sessions passées, ce qui annule la confidentialité persistante apportée par ECDHE. Désactiver les tickets force des poignées de main complètes, avec des clés éphémères.

3. Que garantit HSTS ? Contre quelle attaque protège-t-il ?
Après une première visite, le navigateur refuse toute connexion en HTTP clair vers ce site pendant la durée max-age et n'accepte pas de continuer en cas d'erreur de certificat. Il protège contre le déclassement vers HTTP (SSL stripping) par un attaquant qui se place sur le réseau. Limite : la toute première visite n'est pas protégée, sauf liste de préchargement.

4. Durée maximale des certificats publics : 100 jours en 2027, puis 47 jours en 2029. Conséquences pour une organisation qui renouvelle à la main ?
Le renouvellement manuel devient intenable : plusieurs renouvellements par an et par certificat, avec un risque élevé d'oubli et donc de pannes. Il faut inventorier les certificats, automatiser l'émission et le déploiement (protocole ACME, scripts comme renouveler.sh) et superviser les expirations comme le fait surveiller.py. Une courte durée a un bénéfice : l'exposition d'une clé volée est limitée.

5. Pourquoi un client accepte-t-il une chaîne incomplète et un autre non ?

Certains clients, comme les navigateurs, complètent la chaîne en téléchargeant l'intermédiaire manquant (extension AIA) ou en réutilisant un intermédiaire déjà vu et mis en cache. D'autres (curl, OpenSSL, bibliothèques Java ou Python) ne le font pas et échouent. Il faut donc toujours envoyer la chaîne complète sauf la racine.

6. Procédure si la clé privée du serveur fuit.

(1) Confirmer et circonscrire la fuite, isoler le serveur si nécessaire. (2) Générer une nouvelle clé et une CSR, ne jamais réutiliser l'ancienne. (3) Faire émettre un nouveau certificat et le déployer (renouveler.sh --force). (4) Révoquer l'ancien certificat avec la raison keyCompromise, regénérer la CRL et la redistribuer (crls.pem dans nginx, rechargement). (5) Examiner les journaux pour mesurer l'exposition et changer les secrets qui ont transité par le serveur. (6) Corriger la cause et documenter l'incident.